École Polytechnique de Montréal
Génie mécanique

ROBOT DE RÉADAPTATION PÉDIATRIQUE DES MEMBRES SUPÉRIEUR : GUIDE DE REPRODUCTION ET D'UTILISATION

Polar Lab

Maëlys Carré

Ophélie Sénéchal

Le 5 mars 2026

Table des matières

1 Liste des pièces **1**

2 Impressions 3D **1**

3 Assemblage mécanique **2**

4 Limites et recommandations **5**

5 Électronique et câblage **6**

5.1 Particularités du circuit 6

5.2 Encodeur 7

5.3 Calibration des moteurs 7

5.4 Limites et recommandations 9

6 Cinématique, dynamique et espace de travail **9**

6.1 Cinématique directe 9

6.2 Cinématique inverse 10

6.3 Modélisation dynamique 10

7 Logiciel **11**

7.1 Contrôle de position 13

7.1.1 Architecture du régulateur 13

7.1.2 Suivi de trajectoire 13

7.2 Contrôle par admittance 13

7.2.1 Estimation des forces 14

7.2.2 Loi d'admittance 14

7.2.3 Analyse du comportement de la détection de force 14

7.2.4 Utilisation 15

7.2.5 Limites et améliorations 15

Annexe A. Comparaison avant/après PID 17

Liste des figures

1 Support des engrenages 2

2 Engrenage lié au servomoteur 3
3 Engrenage lié au disque optique 3
4 Partie fixe 4
5 Partie fixe 4
6 Partie fixe 5
7 Partie mobile 5
8 Schéma de câblage pour 1 servomoteur 6
9 7
10 8
11 Schéma du mécanisme RobotRehab 2D 10

Liste des tableaux

1 Liste des pièces - Projet RobotRehab2D 1
2 Résultat de la calibration pour les deux servomoteurs 9

Introduction

Le problème auquel tente de pallier ce présent projet est la paralysie cérébrale chez les enfants, qui englobe un ensemble de conditions ayant comme symptômes la spasticité, l'ataxie ou des mouvements involontaires, et touchant 2 naissances vivantes sur 1000. La kinésithérapie et l'ergothérapie, via la réadaptation, sont un des traitements possibles de cette maladie. (Victorio, 2025) Les directives actuelles recommandent des thérapies axées sur les activités quotidiennes, plutôt que passives (que les patients reçoivent). (McLeod, 2023) En revanche, l'accès à ce type de professionnels est restreint au sein du système de santé public Québécois, se traduisant en de longs délais d'attente, qui peuvent avoir des effets délétères. On compte également des iniquités en ce qui à trait l'accès à ce type de soins, nottament en raison du statut autochtone, de la situation financière ou de la localisation géographique. (Hudon, 2020) Pour pallier cet enjeu, des robots de réadaptation peuvent permettre aux patients d'effectuer certains exercices à la maison. C'est dans cette optique que s'instaure RobotRehab2D, un robot de réadaptation à structure parallèle et à deux degrés de liberté (DDL), à faible coût et à logiciel libre, illustré à la

1 Liste des pièces

Tableau 1 – Liste des pièces - Projet RobotRehab2D

Réf.	Composant	Description & Rôle	Qté	Prix Un.	Total	Lien d'achat
Électronique (E)						
E01	Moteurs Feetech	Servomoteur 7.4V avec encodeur.	2	90,00 $	180,00 $	ca.robotshop.com
E02	Servo 2040	Contrôleur RP2040 (Pimoroni).	2	35,00 $	70,00 $	shop.pimoroni.com
E03	Encodeurs Phidgets	Encodeur optique haute résolution.	2	25,00 $	50,00 $	phidgets.com
E04	Breadboard et fils	Kit d'essai avec fils de connexion.	1	19,99 $	19,99 $	amazon.ca/breadboard
Mécanique (M)						
M01	Tuyau	Bras du mécanisme à 5 barres.	1	31,99 $	31,99 $	amazon.ca/tuyau
M02	Roulement[1]	Roulements 10x28x8 pour les pivots.	3	5,00 $	20,00 $	skf.com/bearing
M03	Visserie	Kit de vis M3/M4 et écrous nylstop.	1	15,88 $	15,88 $	amazon.ca/visserie
M04	Filament 3D	Bobine de 1kg (PLA ou PETG).	1	35,00 $	35,00 $	amazon.ca/filament
Outillage et Fixation (T)						
T01	Serre-joints	Lot de 2 serre-joints Workpro.	2	11,83 $	23,66 $	amazon.ca/workpro
				SOUS-TOTAL	**446,52 $**	
				Taxes estimées (15%)	66,98 $	
				COÛT TOTAL ESTIMÉ (CAD)	**513,50 $**	

[1] Note : Ce roulement ne se vend plus à la date d'écriture de ce document.

2 Impressions 3D

Les modèles 3D au format STL et les fichiers sources SolidWorks sont disponibles sur le dépôt GitHub du projet : GitHub - CAD.

Les composants structurels du robot ont été fabriqués à l'aide d'une imprimante **Bambulab X1-Carbon / P1S** avec système **AMS**, en utilisant du filament **PLA**.

Le tranchage (*slicing*) a été effectué sur le logiciel **BambuStudio** en optimisant le compromis entre précision dimensionnelle et résistance mécanique. Les paramètres généraux appliqués à l'ensemble des pièces sont les suivants :

- **Qualité :** Hauteur de couche de standart Bambulab de $0,20$ mm pour assurer une bonne cohésion inter-couches et un état de surface propre.
- **Résistance :** Densité de remplissage réglée à 40 % avec un motif *Giroïde* (recommandé pour sa résistance multidirectionnelle).
- **Vitesse :** Profil standard Bambulab (vitesse de remplissage allant jusqu'à 200-300 mm/s).
- **Supports :** Activation des supports de type *Arborescent* (*Tree Support*) pour faciliter le retrait et préserver les surfaces fonctionnelles.

Pour les engrenages du système moteur / encodeur, les paramètres de parois ont été augmentés pour garantir la rigidité des dents et éviter la déformation sous charge, passant de 2 parois (par défaut dans la section résistance de BambuStudio) à **6 parois**.

3 Assemblage mécanique

La base a été conçue pour accueillir 2 moteurs Feetech. Les espaces pour les moteurs sont fendus en avant, afin de permettre aux fils électriques reliés aux moteurs de passer. Elle contient deux espaces permettant de la fixer à la table avec les serres-joints.

Par dessus les moteurs se trouve un système permettant d'accueillir les encodeurs. Ces derniers nécessitent une partie mobile, qui tourne en même temps que les moteurs et sur laquelle est fixée le disque optique, et une partie fixe, sur laquelle se trouve le PCB de l'encodeur, permettant la lecture des angles. Le montage de cette solution débute par une pièce qui permettra de laisser passer l'arbre de rotation du servomoteur et qui comportera un arbre de rotation pour le deuxième engrenage :

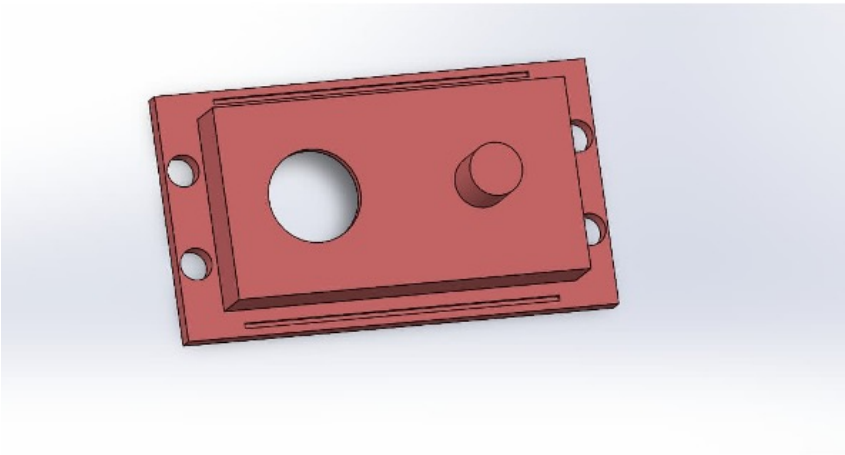


Figure 1 – Support des engrenages

L'arbre du servomoteur est directement lié à un joint ayant une forme d'engrenage :

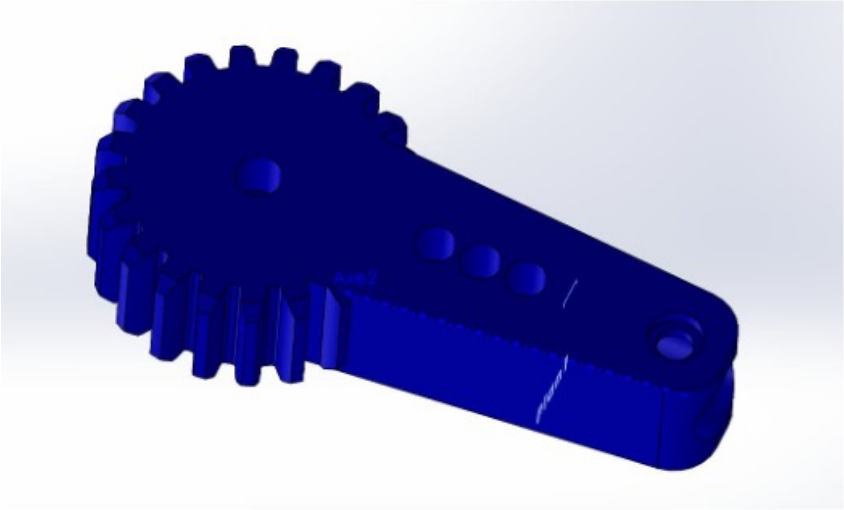


Figure 2 – Engrenage lié au servomoteur

Cet engrenage permet de transmettre le mouvement de rotation à un deuxième engrenage, sur lequel le disque optique sera fixé sur l'arbre de l'engrenage :

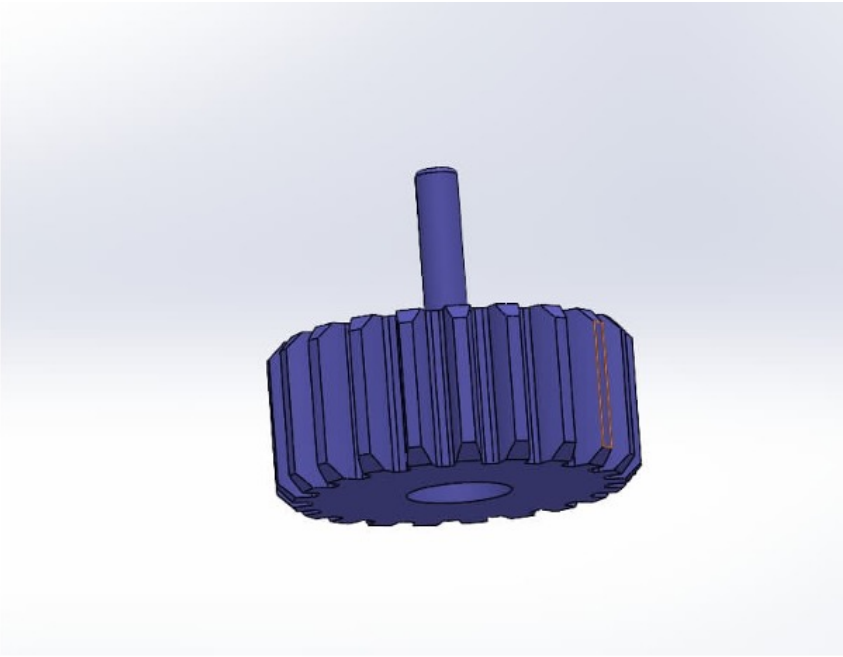


Figure 3 – Engrenage lié au disque optique

Une quatrième pièce, fixée à la pièce rouge, permet de fixer le PCB de l'encodeur :

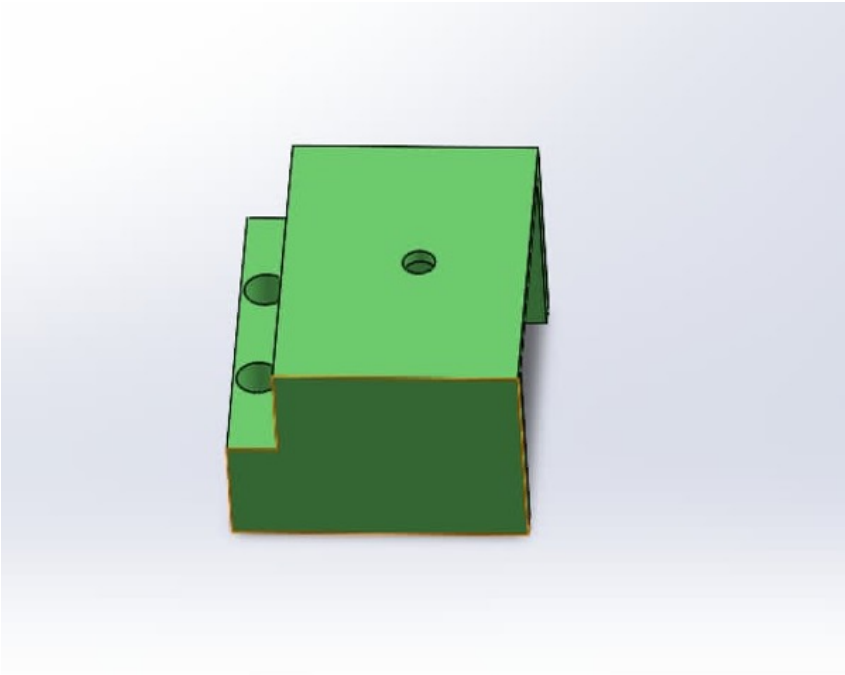


Figure 4 – Partie fixe

La figure suivante permet de mieux comprendre le montage final :

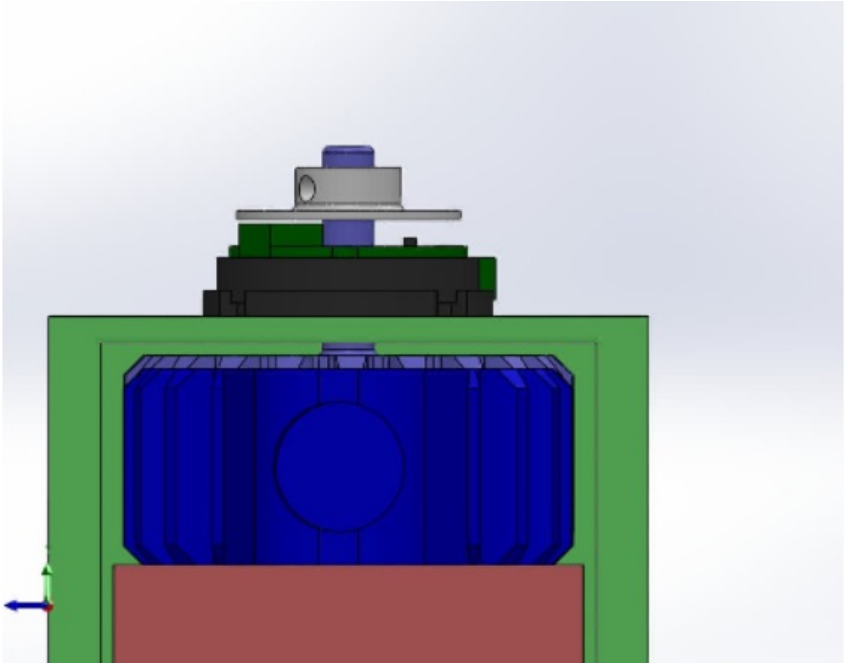


Figure 5 – Partie fixe

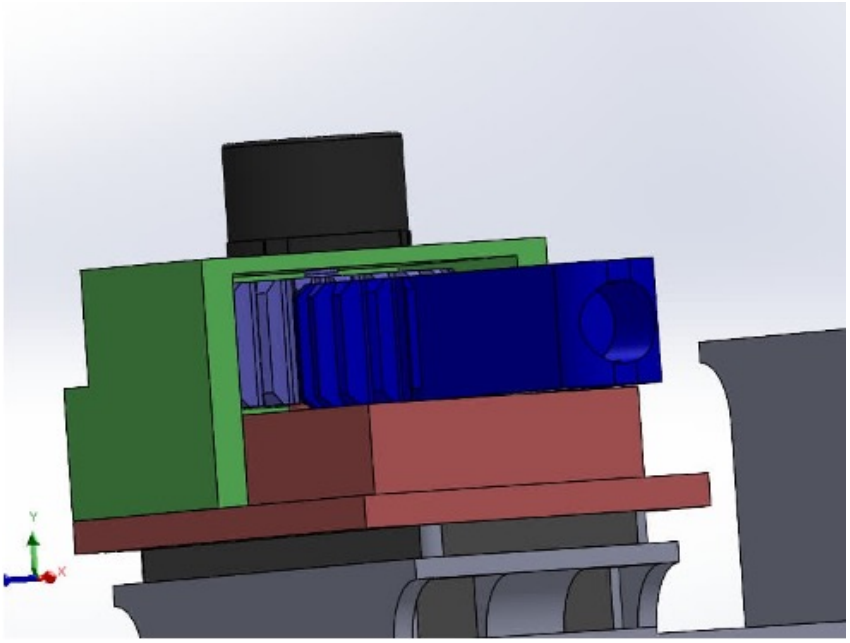


Figure 6 – Partie fixe

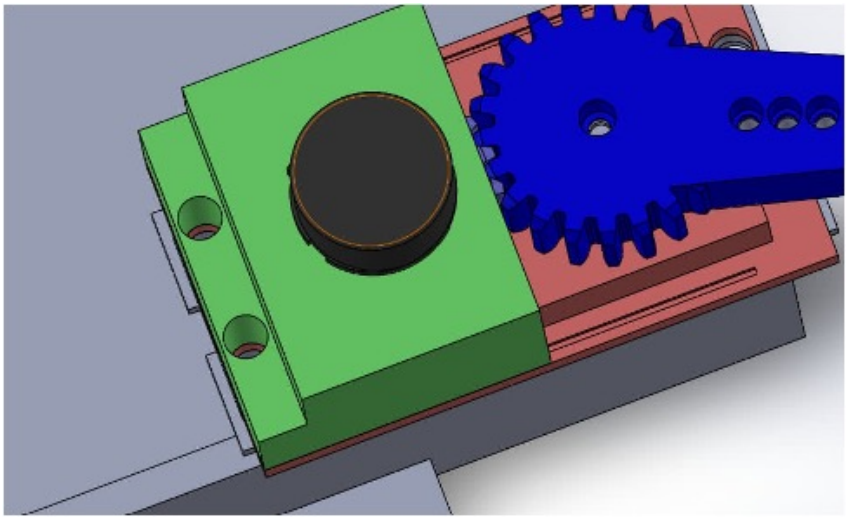


Figure 7 – Partie mobile

4 Limites et recommandations

Des problèmes persistent dans le mécanisme :

— Un jeu apparaît progressivement au niveau des articulations, en raison de la différence de rigidité entre les barres en aluminium et les pièces en PLA. Ce phénomène affecte principalement les liaisons entre le moteur et les barres, ainsi que celles entre la première barre et le mécanisme de roulement. Une amélioration possible serait d'utiliser un matériau plus rigide ou de concevoir un système de fixation complémentaire, par exemple à l'aide de vis, afin de réduire la dépendance aux seuls joints.

— L'asymétrie entre les deux côtés du mécanisme pourrait, à terme, générer un moment entraînant une inclinaison de l'effecteur final. Pour y remédier, il serait envisageable d'intégrer deux roulements à billes côte à côte dans l'effecteur, au lieu d'un seul actuellement, afin d'assurer un alignement parfait des deux bras.

— Les engrenages de l'encodeur sont en contact direct avec leur support, ce qui engendre des frot-

tements indésirables. Une solution consisterait à introduire un léger jeu entre l'engrenage et le support du moteur, ou à réduire la surface de contact en ajoutant, par exemple, un cylindre de petit diamètre qui servirait d'unique point de contact.

— La base avait initialement été conçue pour accueillir l'ensemble des composants électroniques. Toutefois, l'ajout des deux Servo2040 ainsi que des diviseurs de tension pour les encodeurs a rendu l'espace disponible insuffisant.

— Les joints entre les moteurs et la barre (figure 2)se fendent à la moindre force appliquée (donc pendant les algorithmes de contrôle par admittance).

5 Électronique et câblage

Les composantes électroniques clés de RobotRehab2D sont les 2 servomoteurs. Chaque servomoteur est relié à une carte servo2040, qui permet l'envoie des commandes, et un encodeur optique Phidget, qui permet de lire la position du moteur via le mécanisme d'engrenage. Pour chaque moteur, le schéma de câblage est illustré à la figure 8 :

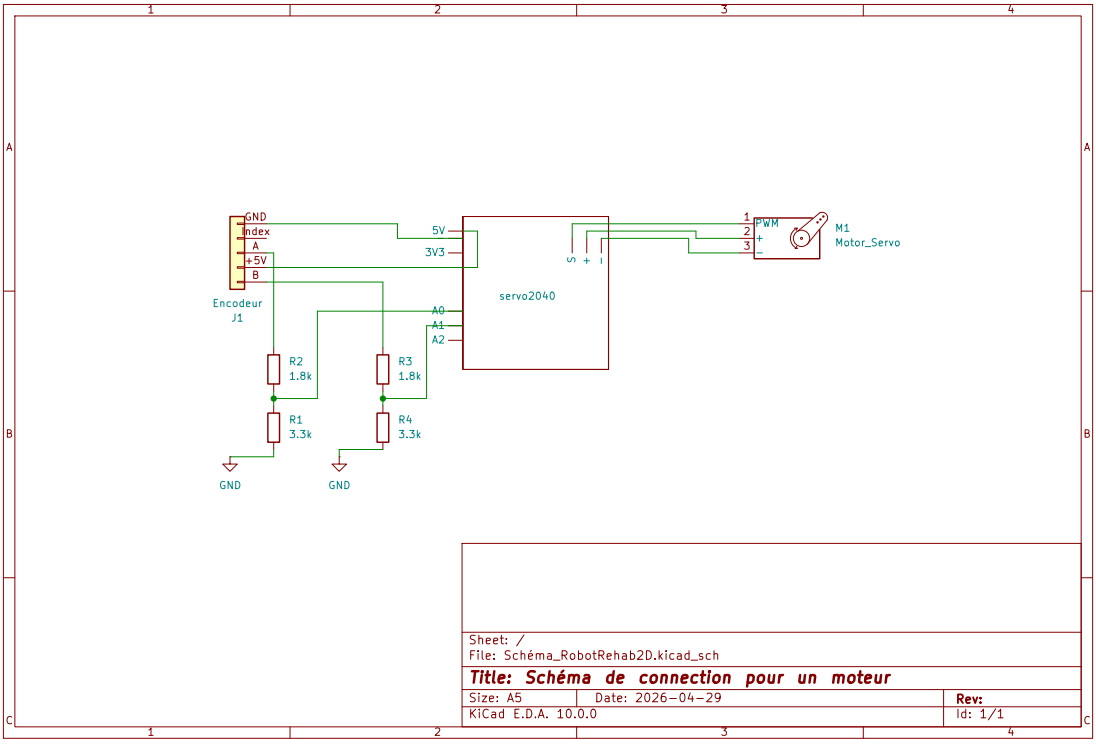


Figure 8 – Schéma de câblage pour 1 servomoteur

5.1 Particularités du circuit

Seulement les pins utilisés ont été représentées sur le schéma de la carte servo2040. Le schéma complet de la carte servo2040 de Pimorini est disponibles à l'adresse suivante : Servo2040 Schematics, et le site de référence est disponible à l'adresse suivante : Website. Il est possible de remarquer qu'un seul moteur est branché à la carte. Ceci est parce que la modalité de lecture de courant de la carte permet de lire le courant consommé par l'ensemble des composants branchés sur la carte. Ainsi, pour lire le courant consommé par un seul moteur, il est nécessaire de brancher un seul moteur par carte. Le courant consommé par l'encodeur sera donc inclut dans la lecture de courant. La lecture de courant permet

d'effectuer la lecture de moment de force appliqué aux moteurs. Cette étape nécessite une calibration afin de faire la correspondance entre le courant des moteurs et le moment de force quils exercent. Cette méthode est décrite à la section 5.3.

Par la suite, du côté de l'encodeur, il est possible de remarquer qu'un diviseur de tension est connecté aux sorties A et B. Ce diviseur de tension permet de transformer le signal sortant de l'encodeur afin qu'il ne dépasse pas le 3.3V, qui est le courant maximal que la carte servo2040 peut recevoir. Des vérifications ont été faites pour s'assurer que l'encodeur devrait réellement être alimenté au 5V. Également, il est possible de remarquer que le signal de l'index n'est pas lu par la carte servo2040. Ce choix est expliqué dans la section 5.2.

5.2 Encodeur

Pour la détection de la position de l'effecteur distal, il est possible d'utiliser un encodeur au niveau des moteurs. Le choix de la technologie s'est arrêté sur l'optique pour sa meilleure précision. Également, un encodeur incrémental plutôt qu'absolu a été choisi en raison des coûts moins élevés. La figure 9 montre le fonctionnement d'un encodeur optique, avec comme composantes clés un émetteur et un récepteur infrarouge, ainsi qu'un disque optique : Le disque optique comporte des fentes, à travers lesquelles la

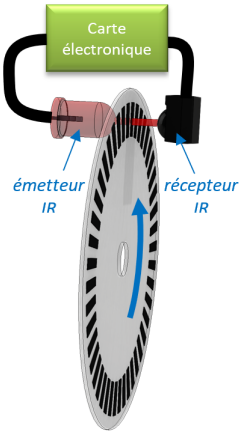


Figure 9

lumière IR peut traverser et être captée par le récepteur. Le résultat est un signal carré. La majorité des encodeurs, incluant celui choisi, comportent 3 signaux : le A, le B et l'index (ou Z). Le signal A et B sont déphasés d'un quart de période et permettent donc de déterminer le sens de rotation de l'encodeur (si la rotation et horaire ou anti-horaire). Le principe de fonctionnement de la détection du sens de la rotation à partir des signaux A et B est montré à la figure 10. Le signal de l'index se produit une fois par tour complet. Pour RobotRehab2D, puisque les servo-moteurs ne devraient jamais effectuer de tour complet, ce signal n'est pas utilisé.

Animation encodeur optique Les encodeurs choisis ont une résolution de 300 CPR (Counts Pre Revolution, et peuvent être retrouvés à l'adresse suivante : Encodeurs Phidgets 300 CPR.

5.3 Calibration des moteurs

La calibration des moteurs permet de trouver l'équivalence entre le courant lu par la carte servo2040 et le moment de force exercée par le moteur. La méthode de calibration est basée sur l'article *Current-Based Impedance Control for Mobile Manipulators Without Force/Torque Sensors* par J.S. de Wolde. Le code pour effectuer la calibration et la visualisation des résultats est disponible sur le Git Hub GBM6710 Servo calibration. Une explication plus détaillée de la méthode et des résultats de la calibration est décrite dans le document $GBM6710_Rapport_projet.pdf$. En résumé, la méthode consiste à envoyer des commandes de balayages de -90° à 90° et de 90° à -90° au servomoteur, avec l'axe de rotation du

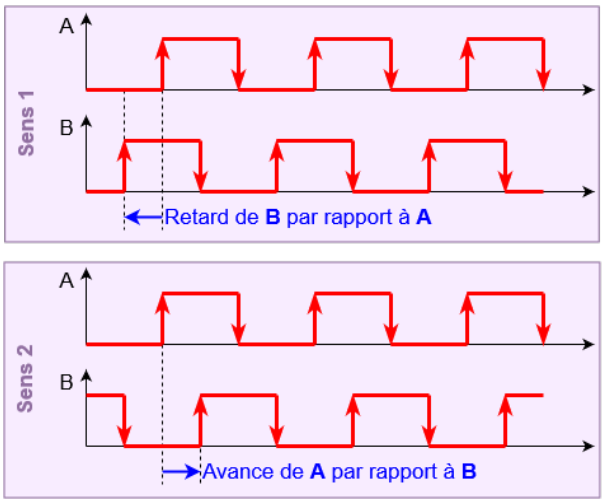


FIGURE 10

moteur perpendiculaire à la gravité, avec le script data_acquisition.py. Un poids devrait être ajouté au bout du bras de levier afin que le bruit des données de courant n'impacte pas trop les résultats.

Cette méthode de calibration est basée sur l'équation dynamique suivante :

$$I\alpha = \tau_{gravity} + \tau_{actuateur} \tag{1}$$

L'article recommande de faire les balayage à vitesse constante afin d'avoir une accélération angulaire nulle et d'obtenir l'équation suivante :

$$\tau_{actuateur} = -\tau_{gravity} \tag{2}$$

Il n'est pas possible de contrôler la vitesse angulaire avec les servomoteurs du Feetechs choisis, cependant en affichant la vitesse angulaire en fonction du temps à partir des données de l'encodeur, la vitesse semble constante durant les balayages. Ainsi, l'approximation d'une accélération angulaire nulle est maintenue.

Le moment de force dû à la gravité varie en fonction de la masse total, de la longueur du bras de levier effectif appliqué au moteur ainsi que de l'angle entre le bras de levier et le vecteur de la gravité :

$$\tau_{gravity} = -mgl \cdot sin(\theta) \tag{3}$$

La fonction compute_COM.py permet de calculer directement la masse et le bras de levier effectif en entrant la masse du poids de calibration ajouté au bout de la barre R1. La position des centre de masse du lien R1 et de l'engrenage du servo ont été obtenu sur SolidWorks, à partir des informations de la modélisation de ces pièces. Ainsi, il est possible de déterminer le moment de force effectif en enregistrant les valeur de courant lu par la carte servo2040 et d'angles lus par l'encodeur. Ces fichiers seront stockés dans un fichiers csv.

Ensuite, le script main_calib.py permet d'effectuer le reste de la méthode de calibration à partir du fichier csv contenant les données des balayages. Tout d'abord, la fonction prepare_data.py permet d'extraire les données durant les balayages seulement, et de les grouper en balayages d'aller (90° vers -90°) et de retour (-90° vers 90°) sous forme de dictionnaire contenant les données de courant, d'angle et de moment de force du modèle (dû à la gravité), et de couper les données correspondant à l'accélération à la ligne 69, puisqu'il y a une accélération et une décélération au début et à la fin du balayage.

Par la suite, la fonction plot_stable_optimisation.py permet d'effectuer le calcul d'optimisation suivant :

$$r^*, f^* = argmin_{r,f} \sum_{i=1}^{N} (I_i - (r \cdot \tau_i + f))^2 \tag{4}$$

Où I_i et τ_i dont le courant mesuré et le moment de force de l'actuateur du modèle respectivement, pour l'échantillon de données i. Le résultat de cette optimisation $r*$ et $f*$ sont les valeurs de ratio

courant/moment de force et de friction (en ampère) pour lesquels la différence entre le moment de force du modèle et le courant mesuré est minimal. Cette fonction permet également de calculer l'erreur quadratique moyenne entre le résultats du moment de force calculé avec les données de calibration et le moment de force du modèle, et d'afficher les résultats intermédiaires.

Une autre méthode d'optimisation est proposée dans l'article afin de minimiser les erreurs du modèle initial, mais cette méthode est dépendante de bonnes données pour les balayages dans les deux directions et n'a pas permis d'obtenir de meilleurs résultats. Toutefois, la fonction plot_stable_COM() a permis d'effectuer cette deuxième optimisation.

Le résultat des calibrations pour les deux servomoteurs est donné au tableau 2.

Servomoteur	r	f
1	0.92	0.62
2	2.011	0.218

Tableau 2 – Résultat de la calibration pour les deux servomoteurs

Les résultats de la calibration sont intégrés dans le script de contrôle du robot pour convertir directement le courant en moment de force.

5.4 Limites et recommandations

Plusieurs défis ont été rencontrés au cours de la réalisation de ce projet.

Bien que les moteurs aient été calibrés individuellement, ils ne fournissent pas un couple moyen uniforme en fonctionnement. Une solution envisageable serait d'intégrer une alimentation externe de 7,4 V, correspondant à la tension recommandée par le fabricant. Jusqu'à présent, les moteurs étaient alimentés via les ports USB d'un ordinateur, soit à 5 V, ce qui peut expliquer ces écarts de performance.

Cependant, l'ajout d'une alimentation externe entraîne certaines contraintes. Selon les recommandations du fabricant du Servo2040, il est nécessaire de sectionner une broche spécifique afin d'éviter d'endommager la carte ainsi que les ordinateurs connectés. De plus, il conviendrait d'étudier les moyens de communication entre les deux microcontrôleurs Servo2040 en l'absence d'un ordinateur.

Il serait également intéressant de remplacer le breadboard par un PCB ou une plaque à trous afin d'avoir un prototype plus robuste. Également, conencter les cartes servo2040 à un autre microcontrôleur, de type Arduino, et de faire la communication entre l'ordinateur et le robot via cet Arduino permettrait de réduire l'encombrement dû aux fils.

6 Cinématique, dynamique et espace de travail

(Hoang, 2015)

6.1 Cinématique directe

$$x_p = l_c + l_a \cos(\theta_4) + l_b \cos\left(2 \arctan\left(\frac{-F + \sqrt{E^2 + F^2 - G^2}}{G - E}\right)\right)$$
$$y_p = l_a \sin(\theta_4) + l_a \sin\left(2 \arctan\left(\frac{-F - \sqrt{E^2 + F^2 - G^2}}{G - E}\right)\right)$$

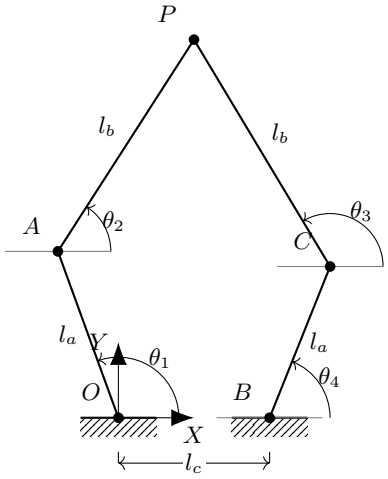


FIGURE 11 – Schéma du mécanisme RobotRehab 2D

Avec :

$$
\begin{aligned}
E &= 2l_b(l_c + l_a(\cos(\theta_4) - \cos(\theta_1))) \\
F &= 2l_a l_b(\sin(\theta_4) - \sin(\theta_1)) \\
G &= l_c^2 + 2l_a^2 + 2l_c l_a \cos(\theta_1) - 2l_a^2 \cos(\theta_4 - \theta_1)
\end{aligned}
$$

Pour la configuration « + - », on prendra + dans le arctan de x_p et − dans le arctan de y_p.

6.2 Cinématique inverse

$$
\begin{aligned}
\theta_1 &= 2\arctan\left(\frac{-F_1 + \sqrt{E_1^2 + F_1^2 - G_1^2}}{G_1 - E_1}\right) \\
\theta_4 &= 2\arctan\left(\frac{-F_4 - \sqrt{E_4^2 + F_4^2 - G_4^2}}{G_4 - E_4}\right)
\end{aligned}
$$

Avec :

$$
\begin{aligned}
E_1 &= -2l_a x_p & F_1 &= -2l_a y_p & G_1 &= l_a^2 - l_b^2 + x_p^2 + y_p^2 \\
E_4 &= 2l_a(-x_p + l_c) & F_4 &= -2l_a y_p & G_4 &= l_c^2 + l_a^2 - l_b^2 + x_p^2 + y_p^2 - 2l_c x_p
\end{aligned}
$$

Pour la configuration « + - », on prendra + dans le arctan de θ_1 et − dans le arctan de θ_4.

6.3 Modélisation dynamique

(Yamine et al., 2020)

$$
\mathbf{J} = \begin{bmatrix} J_{11} & J_{12} \\ J_{21} & J_{22} \end{bmatrix}
$$

où

$$
\begin{aligned}
J_{11} &= \frac{-r_1 s_1}{2} - 2cos(C)\frac{Ar_1c_1 - Br_1s_1}{D} - \frac{E}{B^2}(Br_1c_1 + Ar_1s_1) \\
J_{12} &= \frac{-r_1 s_2}{2} + 2cos(C)\frac{Ar_1c_2 + Br_1s_2}{D} - \frac{E}{B^2}(-Br_1c_2 + Ar_1s_2) \\
J_{21} &= \frac{-r_1 c_1}{2} - 2sin(C)\frac{Ar_1c_1 - Br_1s_1}{D} + \frac{E^{'}}{B^2}(Br_1c_1 + Ar_1s_1) \\
J_{22} &= \frac{-r_1 c_2}{2} + 2sin(C)\frac{Ar_1c_2 + Br_1s_2}{D} + \frac{E^{'}}{B^2}(-Br_1c_2 + Ar_1s_2)
\end{aligned}
$$

Avec

$$
\begin{aligned}
R &= \frac{(l_a + l_b + l_c/2)}{3} \\
r_1, r_2, r_3 &= \frac{l_a}{R}, \frac{l_b}{R}, \frac{l_c/2}{R} \\
s_i &= sin(\theta_1), i = 1, 4 \\
c_i &= cos(\theta_1), i = 1, 4 \\
A &= r_1 \cdot s_1 - r_1 \cdot s_4 \\
B &= 2 \cdot r_3 + r_1 \cdot c_1 + r_1 \cdot c_4 \\
C &= \frac{\pi}{2} + arctan(\frac{A}{B}) \\
D &= 8 \cdot r_2 \cdot \sqrt{1 - \frac{(2B + 2A)}{(4r_2^2)}} \\
E &= \frac{r_2 sin(C)}{1 + \frac{A^2}{B^2}} \sqrt{1 - \frac{A^2 + B^2}{4r_2^2}} \\
E^{'} &= \frac{r_2 cos(C)}{1 + \frac{A^2}{B^2}} \sqrt{1 - \frac{A^2 + B^2}{4r_2^2}}
\end{aligned}
$$

7 Logiciel

La carte servo2040 est programmé en MicroPython. Le mircocontrpoleur sur la carte servo2040 est un RP2040, donc il faut suivre les information relatives à ce dernier. L'environnement de développement utilisé est Thonny. Cet environnement permet de mettre le code sur la carte servo2040 que de communiquer avec cette dernière via Python. Plus d'informations pour commencer à programmer la carte servo2040 sont fournies au lien suivant : *Getting started with Raspberry Pi Pico*.

microcontroleur : get started : appuyer sur le bouton reset en meme temps que brancher à l'ordi, telecharger pico-v1.27.0-pimoroni-micropython.uf2

from https ://github.com/pimoroni/pimoroni-pico/releases

source : https ://github.com/pimoroni/pimoroni-pico/blob/main/setting-up-micropython.md

Une fois les librairies pertinentes installés sur les cartes et l'environnement de programmation installé, il est possible d'installer le code main sur le RP2040 : main.py. Une fois le code installé, les leds de la carte servo2040 devraient s'illuminer lorsque branché. Ce code permet de compter les fronts descendants

de l'encodeur avec la fonction handle_encoder(), de faire la configuration de la lecture de courant de la ligne 52 à 56, et d'envoyer les commandes d'angles au servomoteur avec la fonction aller_a_angle(). Un angle de départ est envoyé à chaque servomoteur, c'est donc la seule ligne qui devrait être différente entre les deux cartes. (ligne 76) À chaque 0.05 secondes, la carte fait l'écoute du port USB, et envoie la commande de position au servomoteur si la commande commence par "S :", et fait la lecture de courant et d'angle de l'encodeur et l'envoie au port d'écoute. La partie effectuée continuellement débute à la ligne 84.

Par la suite, pour contrôler le robot, le script Python PID peut être utilisé. La première étape pour l'utilisation de ce dernier est de connecter un fil à chaque carte, les reliant à l'ordinateur, et de bien nommer le nom des ports séries dans le code. Le nom des ports est à la ligne 180 et 181. À côté de ces lignes est écrit la commande d'angle à appliquer à chaque servomoteur pour réellement atteindre un angle de zéro.Les fonctions pertinentes de ce code sont :

- `cinématique_directe()` qui permet de donner la position de l'effecteur distal en fonction des angles des moteurs.
- La fonction `cinématique_inverse()` donne la position des angles des moteurs pour obtenir une position cible (xc, yc) de l'effecteur distal.
- La fonction `jacobien()` donne la matrice jacobienne pour les angles des servomoteurs θ_1 et θ_4.
- La fonction `dynamique()` permet de calculer la force en x et en y appliquée à l'effecteur distal en fonction du moment de force appliqué à chaque moteur τ_1 et τ_4 et de leur angle θ_1 et θ_4.
- La fonction `current_to_torque()` permet de calculer le moment de force appliqué aux moteurs en fonction des valeurs de ratio courant/moment de force et de friction trouvés dans la calibration.
- La classe `PIDController` est le contrôleur PID de position.
- Les fonctions `initialiser_csv()` et `sauvegarder_donnees()` permettent d'initialiser le fichier CSV et de le remplir avec les données d'angle lues par l'encodeur, de courant mesuré et autres données pertinentes.
- La fonction `tracer_analyse()` permet de tracer les données de position en temps réel (nécessite une carte graphique).
- La fonction `lire_et_afficher()` permet de lire les données d'angle et de courant envoyées par la carte servo2040.
- La fonction `executer_trajectoire()` permet d'effectuer une trajectoire carrée. Il est possible de modifier la variable `sommets` pour changer les coins du carré, ou `nb_pas` pour ajuster la résolution. Pour effectuer plus d'un carré, il faut changer la ligne 242 (paramètre du `range`).
- La fonction `executer_cercle()` suit le même principe mais demande la spécification du centre du cercle.
- L'interface de saisie est définie dans `interface_saisie()` et accepte les commandes "GO", "CERCLE" ou des coordonnées "X Y".

Par la suite, le script plot_jacobien.py permet de lire les données du fichier csv enregistré dans le script PID.py et d'afficher les données. Il débute par séparer les données selon les indices pour séparer chaque cercle/rectangle effectué et affiche la trajectoire de l'effecteur distale, les forces détectés en x et en y et autres données au choix. Les données du déterminant du jacobien sont également affichés, mais sont coupés de -0.2 à 0.2, puis le 2% des valeurs les plus proches de zéro sont affichés en rouge. Ceci permet de confirmer si les pics de valeurs de forces se trouvent bel et bien près des points d'instabilité (lorsque le déterminant de la Jacobienne est près de zéro). et les librairies : lien vers git

Le script `modifier_csv.py` permet de modifier le fichier csv obtenu avec le script PID.py. Pour l'instant, ce scritp permet de mettre les forces à 0 lorsque le déterminant est considéré instable (trop près de 0) et permet de refaire le calcul du moment de force du deuxième moteur avec les données de sa propore calibration.

Le script modélisation_workspace.py permet d'afficher le workspace du robot.

Le Git Hub du projet peut être trouvé à l'adresse suivante : Git Hub de RobotRehab2D. Ce Git Hub contient déjà des données de tests dans le dossier CSV.

7.1 Contrôle de position

Le contrôle de position du robot RobotRehab2D repose sur une architecture en boucle fermée basée sur des régulateurs PID (Proportionnel–Intégral–Dérivé). Cette loi de commande vise à réduire l'écart entre la consigne articulaire, obtenue via la cinématique inverse à partir de la trajectoire cartésienne souhaitée, et la position réelle mesurée par les encodeurs optiques.

Le terme proportionnel agit directement sur l'erreur instantanée afin de corriger rapidement l'écart. Le terme intégral compense les erreurs résiduelles persistantes, notamment dues aux biais mécaniques ou aux frottements. Enfin, le terme dérivé anticipe les variations de l'erreur et permet d'amortir les oscillations, améliorant ainsi la stabilité globale du système.

Cette combinaison permet d'obtenir un suivi de trajectoire plus précis et plus stable, en particulier lors de mouvements continus tels que les trajectoires circulaires. Expérimentalement, l'erreur moyenne de suivi sur une trajectoire circulaire est passée de *6.54 mm sans correction PID* à *2.75 mm avec PID*, et *3.95 mm sans correction PID* à *1.84 mm avec PID* pour une trajectoire rectangulaire, ce qui démontre l'amélioration significative de la précision du système. Les trajectoires avant et après l'implémentation du contrôleur de position PID sont disponibles en Annexe A.

Le code de cet algorithme est disponible dans le dépôt GitHub suivant : `PID.py`.

7.1.1 Architecture du régulateur

Le script implémente une classe `PIDController` basée sur une discrétisation par la méthode de **Tustin** (transformation bilinéaire). Cette approche assure une meilleure stabilité à haute fréquence par rapport à une discrétisation d'Euler classique.

- **Action Proportionnelle (K_p) :** Elle agit sur l'erreur instantanée pour fournir une réponse dynamique rapide.
- **Action Intégrale (K_i) :** Elle élimine l'erreur statique en accumulant les erreurs passées. Le code intègre un mécanisme d'**Anti-Windup conditionnel** qui stoppe l'intégration si la commande atteint la limite de saturation (15.0 unités), évitant ainsi la divergence de la commande lors de blocages mécaniques.
- **Action Dérivée (K_d) :** Elle anticipe les variations de l'erreur pour amortir le mouvement. Elle est couplée à un **filtre passe-bas** ($\alpha = 0.03$) pour limiter l'amplification du bruit de mesure provenant des encodeurs.

7.1.2 Suivi de trajectoire

Pour les mouvements complexes (carrés ou cercles), le script génère des points intermédiaires. À chaque itération de la boucle de contrôle (environ 80 ms), l'algorithme exécute les étapes suivantes :

1. Calcul de la position cartésienne désirée (p_x, p_y).
2. Détermination des angles cibles θ_1 et θ_4 via la fonction `cinematique_inverse`.
3. Calcul des corrections PID individuelles pour chaque moteur afin de compenser les frottements et l'inertie.

7.2 Contrôle par admittance

Le contrôle par admittance permet au robot de réagir aux forces externes exercées par l'utilisateur, créant une interaction haptique. Le principe consiste à simuler un comportement dynamique virtuel (masse-ressort-amortisseur).

7.2.1 Estimation des forces

En l'absence de capteur de force physique à l'effecteur, la force est estimée indirectement via le courant consommé par les servomoteurs :

- **Conversion Courant-Couple :** La fonction `current_to_torque` utilise les paramètres de calibration pour transformer le courant mesuré (I) en moment de force (τ).
- **Modèle Jacobien :** La force à l'effecteur est reliée aux couples moteurs par la transposée de la matrice Jacobienne $J(\theta)$ selon la relation :

$$F = (J^T)^{-1} \cdot \tau \tag{5}$$

  Le code inclut une sécurité vérifiant le déterminant de la Jacobienne ($\det(J) < 0.01$) pour éviter les instabilités numériques près des singularités mécaniques.

7.2.2 Loi d'admittance

Le contrôleur d'admittance a pour objectif de transformer une force externe mesurée en un mouvement cartésien du robot. Contrairement à un contrôle en position classique, où le robot impose sa trajectoire, l'admittance permet ici de modifier la dynamique du système afin de le rendre "compliant", c'est-à-dire qu'il s'adapte aux forces appliquées par l'utilisateur.

Dans cette implémentation, la force externe estimée à partir des couples moteurs est convertie en une vitesse de référence v_r dans l'espace cartésien selon la relation :

$$v_r = \frac{1}{D_e} F_{ext}$$

où D_e représente l'amortissement virtuel. Ce paramètre joue un rôle central dans le comportement du système : une valeur faible rend le robot plus souple et facilement déplaçable, tandis qu'une valeur élevée augmente la résistance au mouvement.

Un second terme intégral est ajouté afin d'améliorer la réactivité sur les sollicitations prolongées et de réduire les effets de biais statiques dans la mesure des forces.

Afin d'assurer la stabilité numérique et de réduire les vibrations dues au bruit de mesure des courants moteurs, un filtre passe-bas est appliqué sur la force estimée. Ce filtrage permet de supprimer les composantes haute fréquence non représentatives des interactions physiques réelles.

Enfin, une zone morte (deadband) est introduite : les forces inférieures à un seuil expérimental sont ignorées afin d'éviter que le robot ne réagisse aux bruits de capteurs ou aux frottements internes du système.

7.2.3 Analyse du comportement de la détection de force

La force externe est estimée à partir des courants moteurs via un modèle simplifié de couple, puis transformée dans l'espace cartésien à l'aide du jacobien du robot. Cette estimation permet de détecter les interactions entre l'utilisateur et le robot sans capteur de force dédié.

En pratique, le système permet de distinguer trois régimes :

- **Zone morte** : aucune action du robot lorsque les forces sont faibles (bruit, frottements).
- **Régime actif** : déplacement proportionnel à la force appliquée.
- **Saturation** : limitation de la vitesse maximale pour garantir la sécurité.

Cette approche permet une interaction naturelle, où le robot ne résiste pas à l'utilisateur mais s'adapte à ses intentions mécaniques.

7.2.4 Utilisation

Le mode d'admittance est activé via l'interface utilisateur en entrant la commande `ADMITTANCE`. Une fois activé, le système passe en boucle fermée et met à jour en continu la position cartésienne en fonction des forces détectées.

L'utilisateur peut alors déplacer manuellement le robot en appliquant une force sur l'effecteur, sans nécessiter de commande explicite en position.

7.2.5 Limites et améliorations

Bien que fonctionnel, le système présente certaines limitations. L'estimation de la force repose sur un modèle indirect basé sur les courants moteurs, ce qui introduit des incertitudes liées aux frottements, aux erreurs de calibration et aux non-linéarités du système. Il pourrait être amélioré en appliquant le filtre passe-bas directement sur les courants et non plus sur la force elle-même.

Par ailleurs, en testant le robot, il est visible que le logiciel actuel comprend l'orientation (x, y) de la force mais pas son intensité, ce qui induit que le robot n'agit pas encore comme une assistance. Une calibration à ce niveau là serait donc nécessaire pour rendre l'interaction adaptée à une utilisation de réadaptation pédiatrique.

Conclusion

Le projet RobotRehab2D a permis de concevoir, implémenter et valider un robot de réadaptation pédiatrique à deux degrés de liberté, basé sur une architecture parallèle de type 5 barres. L'objectif principal consistait à développer une plateforme open-source capable de reproduire des trajectoires cartésiennes tout en intégrant des stratégies de contrôle avancées adaptées aux interactions humaines.

L'ensemble de la chaîne système a été réalisé, allant de la conception mécanique et de la fabrication additive, jusqu'à l'implémentation des lois de commande en temps réel. Le contrôle en position basé sur un régulateur PID a démontré une amélioration significative de la précision de suivi, réduisant les erreurs moyennes de trajectoire de manière importante par rapport à une commande sans correction.

Par ailleurs, l'intégration d'un contrôle par admittance a permis d'introduire une interaction plus naturelle entre l'utilisateur et le robot, en transformant les forces estimées en mouvements cartésiens. Bien que cette approche repose sur une estimation indirecte des forces via le courant moteur, elle constitue une première étape vers une assistance robotisée plus intuitive et adaptable.

Cependant, plusieurs limitations ont été identifiées, notamment liées aux incertitudes de calibration, aux frottements mécaniques, ainsi qu'à la sensibilité du système aux non-linéarités et aux singularités géométriques. Ces aspects ouvrent la voie à des améliorations futures, telles l'optimisation des structures mécaniques ou encore de l'algorithme de contrôle par admittance.

En conclusion, RobotRehab2D constitue une base fonctionnelle et extensible pour des applications de réadaptation à faible coût. Le projet met en évidence le potentiel des systèmes robotiques open-source dans le domaine de la rééducation, tout en soulignant les défis encore présents pour atteindre un niveau clinique robuste et sécurisé.

Références

Gauthier, M. (2025) *Cibles de Santé Québec : des pédiatres craignent pour la qualité des soins aux enfants* https://ici.radio-canada.ca/nouvelle/2206303/compressions-soins-enfants-saguenay-lac-saint-jean

Hoang, M.T. (2015). *Study and Development of Parallel Robots Based On 5-Bar Linkage.* https://www.researchgate.net/publication/283356024_Study_and_Development_of_Parallel_Robots_Based_On_5-Bar_Linkage

Hudon, A. (2020) *Accès insuffisant et inéquitable aux services de réadaptation physique au Québec* https://liguedesdroits.ca/acces-insuffisant-et-inequitable-aux-services-de-readaptation-physique-au-quebec

McLeod, S (2023) *Les soins aux enfants et aux adolescents ayant la paralysie cérébrale (aux niveaux III à V du GMFCS)* https://cps.ca/fr/documents/position/les-soins-aux-enfants-et-aux-adolescents-ayant-la-

Victorio, M.C (2025) *Paralysie cérébrale (infirmité motrice cérébrale)* https://www.merckmanuals.com/fr-ca/professional/pÃľdiatrie/troubles-neurologiques-chez-l-enfant/paralysie-cÃľrÃľbrale-infirmitÃľ-

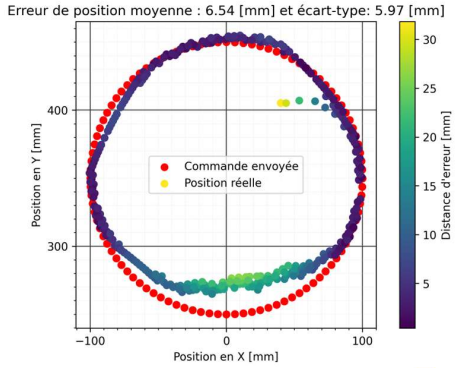
Yamine, J. (2020). *A Planar Parallel Device for Neurorehabilitation.* https://www.mdpi.com/2218-6581/9/4/104

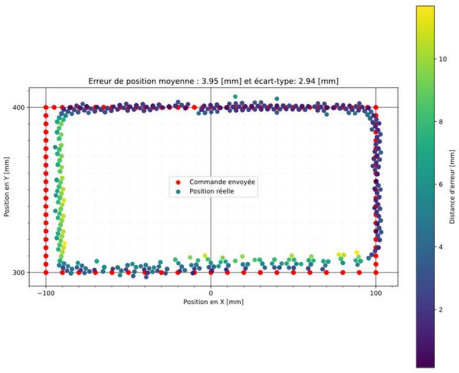
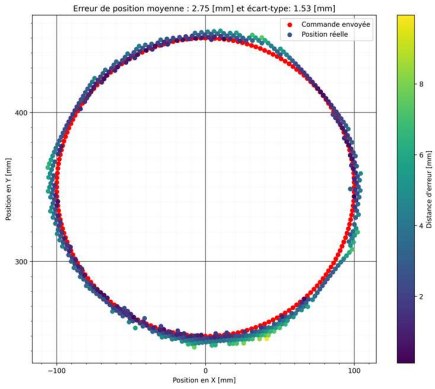
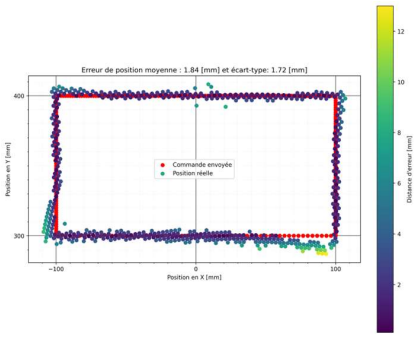
Annexes

Annexe A. Comparaison avant/après PID

Pour retourner au texte, cliquez ici.

- Sans PID





- Avec PID